
Evaluating Redundancy and Compression Limits in Convolutional Neural Networks for Edge Deployment

Sagar Sharma, Kush Lodhi, Vishesh Idnani

Department of Artificial Intelligence & Data Science

Dr. Akhilesh Das Gupta Institute of Professional Studies, New Delhi, India

Under the Guidance of: Ms. Meenu Sharma

Abstract — Deep neural networks deliver high accuracy on image classification benchmarks, yet deploying them on edge hardware remains constrained by model size, inference latency, and memory footprint. This paper presents a systematic empirical study of three compression techniques — unstructured magnitude pruning, structured channel pruning, and post-training static INT8 quantization — applied to ResNet-18 trained on CIFAR-10, with knowledge distillation evaluated as a complementary training strategy. The baseline achieves 95.34% top-1 accuracy at 44.77 MB and 28.2 ms CPU latency. Unstructured pruning demonstrates that 70% of weights are redundant with only 0.44% accuracy loss, but yields no measurable size or latency reduction on standard CPU hardware. Structured channel pruning at 50% removal with extended fine-tuning reaches 94.25% accuracy at 1.62x speedup; combined with post-training static INT8 quantization, this yields the hero result: 94.32% accuracy, 2.07x faster inference, and 2.76 MB — a 15.5x size reduction from baseline with under 1% accuracy cost. Knowledge distillation reveals a capacity-dependent effect: for the 50% pruned model already near its fine-tuning ceiling, distillation provides no benefit (94.11% vs. 94.25%); for the aggressively pruned 70% model, distillation dramatically recovers accuracy from 86.45% to 93.34% (+6.89 percentage points), closing most of the accuracy gap at the cost of structured sparsity. This asymmetric result demonstrates that distillation is most effective when a large accuracy gap exists and the student retains sufficient residual capacity to encode richer supervisory signal. A practical engineering finding is also reported: eager mode static PTQ fails for ResNets due to residual addition dispatch errors; FX graph mode is the correct implementation path. All experiments are fully reproducible via publicly available code and model checkpoints.

Keywords — CNN compression, model pruning, post-training quantization, knowledge distillation, edge deployment, ResNet-18, CIFAR-10.

I. INTRODUCTION

The deployment of convolutional neural networks (CNNs) on edge hardware — Raspberry Pi-class boards, mobile phones, microcontroller units, and embedded systems — is constrained by model size, inference throughput, and memory bandwidth. A standard ResNet-18 contains approximately 11 million parameters, requires over 550 million multiply-accumulate operations (MACs) per forward pass on a 32x32 input, and occupies 44.77 MB at FP32 precision. These characteristics are manageable on server GPUs but present a serious obstacle on CPU-only devices with 256 MB to 8 GB of RAM and no dedicated matrix accelerator.

Model compression addresses this mismatch through three primary mechanisms: pruning (removing redundant parameters), quantization (reducing numerical precision), and knowledge distillation (training a smaller model to mimic a larger one). Each technique targets a different axis of inefficiency. Pruning reduces parameter count; quantization reduces per-parameter storage and arithmetic cost; distillation transfers representational knowledge from a large teacher to a small student. Critically, they are not interchangeable — each has distinct hardware requirements for realizing theoretical efficiency gains in practice.

This paper investigates a specific and practically important question: how far can ResNet-18 be compressed before accuracy meaningfully degrades, and do theoretical efficiency gains translate into measurable CPU inference improvements? The experimental substrate is CIFAR-10, a well-established 10-class benchmark. ResNet-18 is substantially overparameterized for this task — 11.17M parameters for 10 classes — creating the redundancy that compression techniques exploit and producing legible accuracy-efficiency curves.

The theoretical motivation draws from the Lottery Ticket Hypothesis [1], which posits that dense networks contain sparse subnetworks capable of matching full-network accuracy when trained in isolation, and from Deep Compression [2], which demonstrated 35-49x compression on AlexNet and VGGNet via a pruning-quantization-encoding pipeline. This project

reproduces the pruning and quantization stages in a controlled, reproducible setting and extends the analysis to structured pruning and knowledge distillation.

The key contributions of this work are: (1) a clean empirical characterization of unstructured versus structured pruning on ResNet-18, (2) demonstration that structured channel pruning combined with extended fine-tuning and static INT8 quantization achieves 15.5x size reduction and 2.07x speedup with under 1% accuracy cost, (3) evidence that knowledge distillation is capacity-dependent — it provides no benefit over fine-tuning when the model is already near its accuracy ceiling, but recovers 6.89 percentage points on the aggressively pruned 70% model, and (4) an engineering finding regarding PyTorch FX graph mode as the correct quantization path for ResNets with residual connections.

II. RELATED WORK

A. Pruning

Magnitude-based pruning was formalized by Han et al. [3], who demonstrated 9-13x parameter reduction on AlexNet and VGG-16 with no top-5 accuracy loss on ImageNet. Deep Compression [2] extended this to a full three-stage pipeline. The Lottery Ticket Hypothesis [1] reframed pruning as a search problem: dense networks contain sparse winning tickets that match full-network accuracy when identified and retrained. Frankle et al. [20] extended this finding to ResNets with early-checkpoint rewinding. Structured pruning, introduced by Li et al. [6], removes entire filters based on L1-norm, producing real hardware speedup. Fang et al. [18] introduced DepGraph, enabling coupled pruning across skip-connected layers — the foundation of the torch-pruning library used in this work.

B. Quantization

Post-training quantization (PTQ) converts trained FP32 models to lower precision without retraining. Banner et al. [16] demonstrated that calibration-based activation clipping enables INT8 PTQ with minimal accuracy loss. Nagel et al. [17] introduced data-free rounding optimization. PyTorch's native quantization pipeline implements static PTQ using FBGEMM (x86) and QNNPACK (ARM) backends with calibration-based scale estimation.

C. Knowledge Distillation

Knowledge distillation was introduced by Hinton et al. [5] to compress ensemble models. The key insight is that a teacher's soft probability output encodes inter-class similarity information richer than hard labels. Temperature scaling controls distribution softness. Romero et al. [13] extended distillation to intermediate layer matching (FitNets); Park et al. [14] introduced relational knowledge distillation transferring structural relationships between data points.

D. Efficient Architectures

Howard et al. [10] introduced depthwise-separable convolutions in MobileNetV1, reducing parameters by 8-9x. EfficientNet [12] demonstrated compound scaling via neural architecture search. These architectures represent an alternative path to efficiency but are harder to compress further and produce less interpretable compression curves — motivating the choice of overparameterized ResNet-18 as baseline.

III. METHODOLOGY

A. Dataset and Model

All experiments use CIFAR-10 [7]: 50,000 training and 10,000 test images at 32x32 pixels, 10 classes. Standard data augmentation (random horizontal flip, random crop with padding=4) is applied during training; only normalization during evaluation. The baseline model is ResNet-18 [4] adapted for 32x32 input: the 7x7 stride-2 stem convolution is replaced with a 3x3 stride-1 convolution, the maxpool layer is replaced with nn.Identity() to preserve spatial resolution, and the final FC layer outputs 10 classes. This adaptation is standard practice and brings baseline top-1 accuracy above 95%.

B. Baseline Training

ResNet-18 was trained from scratch using SGD with momentum 0.9, weight decay 5e-4, and an initial learning rate of 0.1 with cosine annealing over 200 epochs. Batch size was 128. Training was performed on Google Colab T4 GPU; all inference benchmarks were taken on CPU.

C. Unstructured Magnitude Pruning

Global unstructured magnitude pruning was applied via torch.nn.utils.prune.global_unstructured using L1Unstructured. All Conv2d and Linear weight tensors were included in the global pool. Sparsity levels of 10%, 30%, 50%, 70%, and 90% were

tested. Each pruned model was fine-tuned for 15 epochs with LR 1e-4, SGD, and cosine annealing. Pruning masks remain attached (`prune.remove()` was not called), preserving the sparse forward pass for accuracy measurement.

D. Structured Channel Pruning

Structured pruning was implemented using the torch-pruning library (DepGraph / MagnitudePruner [18]), which constructs a dependency graph of the model and automatically couples pruning across residual branches. L1-norm channel importance was used for global pruning. Channel removal ratios of 30%, 50%, and 70% were tested. The final FC layer was excluded from pruning. Fine-tuning configurations differed by ratio: 30% and 70% used 15 epochs at LR 1e-4; 50% used an extended protocol (40 epochs, LR 1e-3, cosine annealing to 1e-6, CutMix augmentation [21], label smoothing 0.1) following initial results that demonstrated substantial recoverable accuracy.

E. Post-Training Static Quantization

Static INT8 quantization was applied to each structured-pruned model using PyTorch FX graph mode (`torch.ao.quantization, prepare_fx / convert_fx`) with the fbgemm backend. Eager mode PTQ was attempted first and failed with `aten::add.out` is not implemented for QuantizedCPU — the residual addition operations in ResNet skip connections require explicit `FloatFunctional` wrappers in eager mode, which cannot be inserted into torchvision's standard ResNet without model rewriting. FX graph mode resolves this automatically by tracing the full computation graph. Calibration was performed on approximately 512 CIFAR-10 training images (8 batches of 64).

F. Knowledge Distillation

A ResNet-50 teacher was trained from scratch on CIFAR-10 (same adaptation as baseline) for 200 epochs, reaching 93.46% validation accuracy. The structured-50% pruned student was initialized from its fine-tuned checkpoint. Distillation training ran for 40 epochs with LR 1e-3, SGD, cosine annealing, temperature $T=4.0$, and loss weighting $\alpha=0.7$ (soft KL loss) + 0.3 (hard cross-entropy). The combined distillation loss follows [5]:

$$L = \alpha * T^2 * KL(\text{softmax}(z_s/T) || \text{softmax}(z_t/T)) + (1 - \alpha) * CE(z_s, y)$$

G. Evaluation Metrics

Top-1 accuracy was evaluated on the full CIFAR-10 test set (10,000 images). Inference latency was measured as the mean of 100 single-image forward passes on CPU, preceded by 20 warmup runs, using `time.perf_counter`. Peak RAM was recorded using `tracemalloc`. Model size was the on-disk checkpoint file size. MACs were computed using torch-pruning's `count_ops_and_params` utility. All latency comparisons within a session are valid; cross-session absolute values reflect Google Colab instance variability and should be interpreted cautiously.

IV. EXPERIMENTAL RESULTS

A. Baseline

The CIFAR-adapted ResNet-18 trained from scratch achieves 95.34% top-1 accuracy at 44.77 MB and 28.2 ms CPU inference latency. Parameter count is 11.17M; MACs are 557.2M per forward pass on a 32x32 input. These figures serve as the reference for all compression comparisons.

TABLE I: Baseline ResNet-18 Benchmarks

Metric	Value
Top-1 Accuracy	95.34%
Model Size (FP32)	44.77 MB
Parameters	11.17 M
MACs	557.2 M
CPU Inference Latency	~28.2 ms

B. Unstructured Magnitude Pruning

Global unstructured magnitude pruning results are shown in Table II. Accuracy is stable through 70% sparsity, degrading by only 0.44 percentage points. At 90% sparsity, accuracy collapses to approximately 42% — clearly past the operational threshold. Critically, model size and latency remain completely flat across all sparsity levels. Unstructured pruning masks weights in-place via multiplicative masks; no weights are physically removed, so tensor dimensions and memory layout are

unchanged. Standard x86 CPUs do not implement sparse matrix-vector multiply for general-purpose neural network operations, so the zero weights are computed at full cost.

TABLE II: Unstructured Pruning Results

Sparsity	Accuracy	Size (MB)	Latency (ms)
0% (Baseline)	95.34%	44.77	~28.2
10%	~95.3%	44.77	~28
30%	~95.2%	44.77	~28
50%	~95.1%	44.77	~28
70%	~94.9%	44.77	~28
90%	~42.0%	44.77	~28

This result constitutes an empirical confirmation of a well-known theoretical limitation: unstructured sparsity does not produce hardware speedup on commodity CPUs without native sparse compute support. The finding directly motivates the transition to structured pruning.

C. Structured Channel Pruning

Structured pruning results are shown in Table III. Unlike unstructured pruning, tensor dimensions physically shrink when channels are removed, producing real reductions in both model size and inference latency proportional to the reduction in MACs. The 50% model with extended fine-tuning recovers to 94.25% — only 0.99 percentage points below baseline. The 70% model reaches 4.71x speedup but loses 8.79 percentage points of accuracy.

TABLE III: Structured Pruning Results

Model	Accuracy	Latency (ms)	Size (MB)	Params (M)	MACs (M)	Speedup
Baseline	95.24%	21.48	42.70	11.17	557.2	1.00x
Struct-30%	93.31%	18.62	20.89	5.46	269.8	1.15x
Struct-50%	94.25%	13.30	10.73	2.80	140.2	1.62x
Struct-70%	86.45%	4.56	3.85	1.00	50.0	4.71x

A notable finding is the role of fine-tuning protocol. The structured-50% model with standard fine-tuning (15 epochs, LR 1e-4) reached 91.41%; with extended fine-tuning (40 epochs, LR 1e-3, CutMix, label smoothing) it reached 94.25% — a 2.84 percentage point recovery. This demonstrates that fine-tuning protocol choice is a major determinant of post-compression accuracy, and that practitioners routinely leave accuracy on the table with generic configurations.

D. Post-Training Static INT8 Quantization

Static INT8 quantization was applied to all three structured-pruned models. Results are in Table IV. Quantization introduces less than 0.15 percentage points of accuracy change across all models — negligible and within measurement noise for a 10,000-sample test set. The 4x storage reduction is consistent across all sizes, as expected from the FP32-to-INT8 precision reduction. The latency benefit of INT8 shows clear diminishing returns with model size: 1.77x additional speedup at 30% pruning (5.46M params), 1.28x at 50% pruning (2.80M params), and only 1.14x at 70% pruning (1.00M params). At very small model sizes, fixed overheads — PyTorch dispatch, kernel launch, Python interpreter — dominate inference time, and memory bandwidth is no longer the bottleneck that INT8 is designed to address.

TABLE IV: Static INT8 Quantization Results

Model	Acc FP32	Acc INT8	Lat FP32	Lat INT8	Size FP32	Size INT8	Speedup vs. Baseline
Struct-30%	93.31%	93.21%	18.62 ms	10.55 ms	20.89 MB	5.30 MB	2.04x
Struct-50%	94.25%	94.32%	13.30 ms	10.39 ms	10.73 MB	2.76 MB	2.07x
Struct-70%	86.45%	86.40%	4.56 ms	3.99 ms	3.85 MB	1.02 MB	5.38x

Hero result: Structured-50% pruning (extended fine-tune) + static INT8 quantization achieves 94.32% accuracy, 2.07x faster inference, and 2.76 MB on-disk footprint — a 15.5x total size reduction from the 44.77 MB FP32 baseline, with under 1% accuracy cost.

TABLE V: Combined Compression Stack (Struct-50% + INT8)

Stage	Accuracy	Latency (ms)	Size (MB)	Speedup
Baseline (FP32)	95.24%	21.48	42.70	1.00x
+ Structured 50% Pruning	94.25%	13.30	10.73	1.62x
+ Static INT8 Quantization	94.32%	10.39	2.76	2.07x

E. Knowledge Distillation

Knowledge distillation experiments were conducted on two student configurations: the structured-50% model (initialized from the extended fine-tune checkpoint, 94.25% baseline accuracy) and the structured-70% model (initialized from its fine-tuned checkpoint, 86.45% baseline accuracy). The same distillation configuration was used for both: ResNet-50 teacher, 40 epochs, LR 1e-3, SGD with cosine annealing, temperature $T=4.0$, $\alpha=0.7$. Results are shown in Table VI.

TABLE VI: Knowledge Distillation Results

Model	Acc Before Distillation	Acc After Distillation	Delta	Size INT8
Struct-50% (fine-tuned)	94.25%	94.11%	-0.14%	2.76 MB
Struct-70% (fine-tuned)	86.45%	93.34%	+6.89%	1.02 MB

The two results reveal a capacity-dependent effect of distillation that is both interpretable and practically significant. For the structured-50% model, distillation produces 94.11% — a 0.14 percentage point difference from the fine-tuned baseline of 94.25%, well within the 95% confidence interval of approximately $\pm 0.4\%$ for a 10,000-sample test set. The null result here reflects a ceiling constraint: the model has 2.80M parameters and has already been pushed near its representational capacity by extended fine-tuning with CutMix and a well-tuned learning rate schedule. A richer training signal from the teacher cannot recover channels that were physically removed during pruning.

For the structured-70% model, distillation produces a dramatically different outcome: accuracy rises from 86.45% to 93.34%, a +6.89 percentage point improvement. This model has 1.00M parameters and only 50.0M MACs — severely compressed, with a correspondingly large accuracy gap relative to baseline. Two factors explain why distillation succeeds here. First, the gap between 86.45% and the fine-tuned ceiling is large enough that there is genuine recoverable accuracy — the model is not already near its capacity limit. Second, despite being heavily pruned, the 70% model retains sufficient representational capacity to encode the inter-class similarity structure communicated by the teacher's soft probability distributions. The teacher's signal provides substantially richer gradient information than hard labels alone, and this matters more when the student is far from its accuracy ceiling.

The asymmetry between the two results gives a practical rule: distillation is most valuable when (a) there is a large accuracy gap between the fine-tuned compressed model and the baseline, and (b) the model has not already been pushed to its representational ceiling by a high-quality fine-tuning protocol. When fine-tuning has already achieved near-optimal accuracy for the given capacity, distillation adds marginal value at the cost of a full training run with a teacher.

V. DISCUSSION

A. Research Questions Revisited

RQ1 — How much redundancy exists in ResNet-18 on CIFAR-10? Approximately 70% of weights are redundant under unstructured pruning, and 50% of channels can be removed under structured pruning with under 1% accuracy cost after extended fine-tuning. The ratio of 11.17M parameters to 50,000 training examples (223:1) and 10 classification targets is indicative of severe overparameterization for this task.

RQ2 — At what compression ratio does accuracy degrade significantly? For unstructured pruning, the inflection lies between 70% and 90% sparsity. For structured pruning, the inflection is between 50% and 70% channel removal — 50%

maintains under 1% accuracy degradation while 70% produces an 8.79 percentage point loss. However, the 70% model's accuracy gap is largely recoverable via distillation (86.45% to 93.34%), suggesting the practical compression limit is more gradient-dependent than a single number implies.

RQ3 — Do theoretical efficiency gains translate to CPU improvements? For unstructured pruning: no. For structured pruning: yes, proportional to channel reduction. For INT8 quantization applied to structured models: yes for size (consistently $\sim 4\times$), and yes for latency at larger model sizes with diminishing returns below $\sim 2\text{M}$ parameters.

B. Distillation as a Gap-Recovery Tool

The distillation results reveal a principle not clearly articulated in the standard literature: knowledge distillation's effectiveness depends on the ratio of remaining accuracy gap to remaining model capacity, not on absolute model size alone. The 70% model at 1.00M parameters benefits enormously (+6.89 pp) precisely because it has a large gap (8.79 pp below baseline) and enough residual capacity in its surviving channels to encode the teacher's richer inter-class signal. The 50% model at 2.80M parameters sees no benefit because fine-tuning has already closed most of the gap — distillation arrives when there is little left to recover.

A practical implication: when applying aggressive structured pruning ($>60\%$ channel removal), distillation should be the default recovery strategy rather than extended fine-tuning, as the student is far from its accuracy ceiling and the teacher's soft targets provide a proportionally larger signal gain over hard labels.

C. Engineering Findings

Practitioners attempting to apply static PTQ to ResNets using PyTorch's eager mode API will encounter the `aten::add.out` is not implemented for QuantizedCPU error on residual addition operations. This is not documented prominently in existing tutorials. FX graph mode (`torch.ao.quantization` with `prepare_fx` / `convert_fx`) is the correct solution and should be the default approach for any model with residual connections.

The ordering of compression techniques matters. Structured pruning should precede quantization: INT8 quantization of a dense FP32 model reduces storage but provides limited latency benefit at this scale, while quantizing an already structurally compressed model yields meaningful additional speedup.

VI. CONCLUSION

This paper presented a systematic empirical evaluation of CNN compression for edge deployment, applied to ResNet-18 on CIFAR-10. The principal finding is that a combination of structured channel pruning (50% removal, extended fine-tuning) and post-training static INT8 quantization achieves a 15.5x size reduction and 2.07x inference speedup with under 1% accuracy cost. Unstructured pruning, while demonstrating high theoretical redundancy (70% of weights expendable), provides no hardware speedup on standard CPUs. Knowledge distillation reveals a capacity-dependent effect: it adds no benefit over fine-tuning for the 50% pruned model already near its accuracy ceiling, but recovers 6.89 percentage points on the aggressively pruned 70% model — bringing it from 86.45% to 93.34% and making extreme compression practically viable for accuracy-tolerant deployments.

The practical guidance that emerges is a two-path recommendation. For accuracy-critical deployments, apply structured pruning at 30-50% channel removal, invest in fine-tuning protocol optimization (CutMix, higher LR, cosine annealing), and follow with static INT8 quantization via PyTorch FX graph mode. For latency-critical deployments tolerating moderate accuracy cost, apply aggressive 70% structured pruning and use knowledge distillation rather than extended fine-tuning as the recovery strategy; the resulting 1.00M parameter INT8 model runs at 3.99 ms and 1.02 MB.

Future work includes quantization-aware training on compressed models, layer-wise adaptive pruning based on sensitivity analysis, and real hardware evaluation on Raspberry Pi or mobile devices using ONNX Runtime or TensorFlow Lite. Extending to CIFAR-100 would test the generalizability of the capacity-dependent distillation finding to harder classification tasks.

REFERENCES

- [1] J. Frankle and M. Carbin, "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks," ICLR, 2019.
- [2] S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding," ICLR, 2016.
- [3] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both Weights and Connections for Efficient Neural Networks," NeurIPS, 2015.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016.
- [5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," NeurIPS Deep Learning Workshop, 2015.
- [6] H. Li et al., "Pruning Filters for Efficient ConvNets," ICLR, 2017.
- [7] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," Technical Report, University of Toronto, 2009.

-
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," NeurIPS, 2012.
 - [9] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," ICLR, 2015.
 - [10] A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.
 - [11] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
 - [12] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019.
 - [13] A. Romero et al., "FitNets: Hints for Thin Deep Nets," ICLR, 2015.
 - [14] J. Park, S. Kim, and J. Lee, "Relational Knowledge Distillation," CVPR, 2019.
 - [15] M. Nagel et al., "Data-Free Quantization through Weight Equalization and Bias Correction," ICCV, 2019.
 - [16] R. Banner, Y. Nahshan, and D. Soudry, "Post Training 4-bit Quantization of Convolutional Networks for Rapid Deployment," NeurIPS, 2019.
 - [17] C. Fang et al., "DepGraph: Towards Any Structural Pruning," CVPR, 2023.
 - [18] Y. LeCun, J. S. Denker, and S. A. Solla, "Optimal Brain Damage," NeurIPS, 1990.
 - [19] J. Frankle et al., "Linear Mode Connectivity and the Lottery Ticket Hypothesis," ICML, 2020.
 - [20] S. Yun et al., "CutMix: Training Strategy that Makes Use of Sample and the Training Labels," ICCV, 2019.
 - [21] PyTorch Documentation, "torch.ao.quantization — FX Graph Mode Quantization," 2023. [Online]. Available: <https://pytorch.org/docs/stable/quantization.html>
 - [22] torch-pruning Documentation, "DepGraph: Structural Pruning for PyTorch," GitHub, 2023. [Online]. Available: <https://github.com/VainF/Torch-Pruning>